

Übungsblatt 9: Capstone-Projekt und Timing-Verifikation

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

Willkommen zum großen Finale! Im *Capstone-Projekt* fügen Sie alle Konzepte des Semesters zusammen. Sie entwickeln die Firmware für einen fiktiven, smarten Industrie-Sensor auf dem **STM32H755**.

Das System ist asymmetrisch (**AMP**) aufgebaut: Der Cortex-M4 (FreeRTOS) liest zeitkritisch und jitterfrei Hardware-Impulse via Interrupt ein, sichert sich selbst mit einem **Window Watchdog (WWDG)** ab und übergibt die Daten über einen geteilten Speicher (**SRAM4**) an den Cortex-M7. Der M7-Kern berechnet aufwendige Fließkomma-Filter und übernimmt die langsame UART-Ausgabe. Die Synchronisation zwischen den Kernen erfolgt atomar über **Hardware-Semaphoren (HSEM)**.

Im letzten Schritt verifizieren Sie die End-to-End-Latenz und die Jitterfreiheit Ihres Cortex-M4-Kerns mit der **TASKING BlueBox**.

Aufgabe 1: Systemkonfiguration im CubeMX

Bereiten Sie das Dual-Core-Projekt für beide Kerne vor. Gehen Sie sorgfältig anhand dieser Checkliste vor:

1. **Dual-Core und OS:** Aktivieren Sie CM7 und CM4. Aktivieren Sie FreeRTOS (**CMSIS_V2**) *ausschließlich* für den CM4.
2. **Hardware-Interrupt (M4):** Konfigurieren Sie den blauen User-Button (PC13) als GPIO_EXTI13. Aktivieren Sie den EXTI-Interrupt im NVIC.
3. **WWDG (M4):** Aktivieren Sie den Window Watchdog auf dem CM4 (inkl. EWI-Interrupt im NVIC; Parameter analog zu Übungsblatt 8).
4. **HSEM:** Aktivieren Sie das HSEM-Modul und weisen Sie es **beiden** Kernen zu.
5. **FreeRTOS-Objekte (M4):** Legen Sie eine Queue (**SensorQueue**, Message Size **uint32_t**) und eine Task (**M4_SensorTask**, Priorität **osPriorityHigh**) an.
6. **UART (M7):** Aktivieren Sie einen USART (z. B. USART3) für die Terminal-Ausgabe auf dem M7.
7. Generieren Sie den Code. Shared Memory (**sd / shared_data.h** im SRAM4) und MPU-Konfiguration übernehmen Sie aus Termin 6.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Die Architektur trennt harte und weiche Echtzeit strikt. Der M4 wird durch FreeRTOS und EXTI zum hochdeterministischen Sensor-Interface. Der M7 bleibt ein „Zahlenfresser“ (Bare-Metal) mit Anbindung an die langsame Außenwelt (UART) — bewusst *ohne* FreeRTOS, damit `printf`/UART den harten M4-Pfad nicht belasten können (vgl. Termin 7).

Das Setup der HSEM ist zwingend für beide Kerne erforderlich, da sonst die CubeIDE den HAL-Treiber für die Bus-Sperre nicht in beide Projektordner generiert. Die Queue ersetzt den binären Semaphor aus Übungsblatt 5: Sie transportiert zusätzlich einen Zeitstempel (*Copy by Value*).

Aufgabe 2: Hard Real-Time Domain (Cortex-M4)

Implementieren Sie die Firmware für den Echtzeit-Kern. Nutzen Sie das Muster *Deferred Interrupt Processing* (vgl. Übungsblatt 5).

1. **Top-Half (ISR):** Überschreiben Sie in der `main.c` des M4 den EXTI-Callback. Holen Sie einen Zeitstempel (`HAL_GetTick()`) und legen Sie ihn mittels `osMessageQueuePut()` in die `SensorQueue`. Nutzen Sie in der ISR zwingend den Timeout 0!
2. **Bottom-Half (Task):** Die `M4_SensorTask` liest die Queue mit einem Timeout von 15 ms aus (nicht `osWaitForever` — siehe Hinweis unten).
3. Sobald ein Zeitstempel empfangen wird, fordern Sie das Hardware-Schloss `HSEM_0` an (Try-Lock mit `osDelay(1)`-Polling, wie in Übungsblatt 8).
4. Schreiben Sie den Zeitstempel in das Shared-Memory-Struct im SRAM4 (`sd.payload_value`; Linker-Skript und MPU aus Termin 6 vorausgesetzt).
5. Geben Sie das HSEM frei.
6. Füttern Sie den WWDG am Ende *jedes* Task-Schleifendurchlaufs korrekt im erlaubten Zeitfenster.

Hinweis zum Queue-Timeout: Ein blockierendes `osWaitForever` würde die Task nur bei Tastendruck wecken — der WWDG bliebe ohne Button-Events ungefüttert und würde das System zurücksetzen. Der Timeout von 15 ms erlaubt einen periodischen Schleifendurchlauf zum Watchdog-Refresh auch ohne Sensor-Event und liegt im *Allowed Window* der WWDG-Parameter aus Übungsblatt 8 (dort führte 50 ms bereits in die Late Phase).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Top-Half (ISR):

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == USER_Btn_Pin) {
        uint32_t timestamp = HAL_GetTick();
        /* Timeout 0: in der ISR niemals blockieren */
        osMessageQueuePut(SensorQueueHandle, &timestamp, 0U, 0U);
    }
}

```

Besonderheit von CMSIS in der ISR:

Wie in Übungsblatt 5 liest der CMSIS-Wrapper das IPSR-Register. Im Handler Mode leitet `osMessageQueuePut()` intern auf die sichere `...FromISR`-Variante um und fordert bei Bedarf `PendSV` an. In der nativen FreeRTOS-API müssten Sie `xQueueSendFromISR()` und `portYIELD_FROM_ISR` manuell verwenden.

Bottom-Half (M4_SensorTask):

```

void StartM4SensorTask(void *argument)
{
    uint32_t rx_time;

    for (;;) {
        /* 1. Warten auf Daten aus der Top-Half - Timeout 15 ms:
         * erlaubt WWDG-Refresh auch ohne Tastendruck */
        if (osMessageQueueGet(SensorQueueHandle, &rx_time, NULL, 15) == osOK) {

            /* 2. Hardware-Schloss anfordern (kein Busy-Spin unter FreeRTOS!) */
            while (HAL_HSEM_FastTake(0) != HAL_OK) {
                osDelay(1);
            }

            /* 3. Shared Memory aktualisieren (Critical Section) */
            sd.payload_value = rx_time;

            /* 4. Schloss freigeben */
            HAL_HSEM_Release(0, 0);
        }

        /* 5. Watchdog im Allowed Window füttern (jeder Loop-Durchlauf) */
        HAL_WWDG_Refresh(&hwwdg);
    }
}

```

Die Task verbraucht ohne Queue-Event nahezu 0% CPU (Blocked auf dem Queue-Get). Trifft innerhalb von 15 ms kein Event ein, kehrt `osMessageQueueGet` mit Timeout zurück — die Schleife füttert trotzdem den WWDG und blockiert erneut.

Aufgabe 3: Soft Real-Time Domain (Cortex-M7)

Implementieren Sie die Firmware für den Master-Kern.

1. Befreien Sie den M4 in der `main.c` des M7 aus dem Reset (`HAL_PWREx_ReleaseCore`).
2. Deaktivieren Sie den Cache für den SRAM4-Bereich über die MPU (*Disable Cache*), um Kohärenzfehler zu vermeiden (vgl. Übungsblatt 6).
3. Fordern Sie in der Super-Loop das `HSEM_0` an. Lesen Sie den Wert aus dem SRAM4. Geben Sie das Schloss *sofort* wieder frei.
4. Führen Sie erst *nach* der Freigabe des Schlosses eine Float-Filterung durch (simulieren Sie diese durch eine Rechenschleife oder ein `HAL_Delay(10)`).
5. Geben Sie das Ergebnis über UART via `printf()` aus.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* 1. MPU konfigurieren (SRAM4 Non-Cacheable) und M4 aufwecken */
    MPU_Config();
    HAL_PWREx_ReleaseCore(PWR_CORE_CPU2);

    uint32_t sensor_val = 0;

    while (1) {
        /* 2. Hardware-Schloss anfordern (Polling im Bare-Metal ok) */
        if (HAL_HSEM_FastTake(0) == HAL_OK) {
            sensor_val = sd.payload_value;
            HAL_HSEM_Release(0, 0); /* Sofort wieder freigeben! */

            /* 3. Soft-Realtime-Arbeit außerhalb der Critical Section */
            float filtered = (float)sensor_val * 1.5f;
            HAL_Delay(10); /* Simuliere Rechenzeit */
            printf("Gefilterter Sensorwert: %f\n", filtered);
        }
    }
}
```

Didaktischer Kern: Die Critical Section (HSEM gehalten) bleibt so kurz wie möglich — nur das Lesen des Payloads. Filter und `printf` laufen *danach*. Damit blockiert der weiche M7 den harten M4-Pfad nicht länger als nötig. `printf`/UART gehören bewusst ausschließlich auf den Soft-Echtzeit-Kern.

Aufgabe 4: Timing-Verifikation mit der BlueBox

Beweisen Sie nun die Echtzeit-Fähigkeit Ihrer Architektur mit dem Hardware-Tracer (vgl. Übungsblatt 7).

1. Starten Sie das System und verbinden Sie die **TASKING BlueBox**. Laden Sie in winIDEA die `.elf`-Dateien beider Kerne.
2. Messen Sie im Profiler die End-to-End-Latenz: die Zeit vom EXTI-Hardware-Event (Tastendruck) bis zum Freigeben des HSEM-Schlusses im M4.
3. Führen Sie eine Jitter-Analyse durch: Betrachten Sie die Aufwachzeitpunkte der `M4_SensorTask`. Zeigt das M4-Timing Schwankungen (Jitter), während der M7 im Hintergrund rechnet oder auf den UART wartet?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

End-to-End-Latenz:

Durch das Deferred Interrupt Processing ist die Hardware-ISR extrem kurz. Die Queue weckt die `M4_SensorTask` sofort auf (Context Switch in wenigen Mikrosekunden). Die gemessene End-to-End-Latenz vom Tastendruck bis zum beschriebenen Shared Memory sollte konstant im sehr niedrigen Mikrosekundenbereich liegen.

Jitter-Analyse (der finale Beweis):

Im Gantt-Chart des Tracers wird sichtbar, dass der Cortex-M4 völlig unbeeindruckt und präzise arbeitet (Jitter ≈ 0). Das ist der enorme Vorteil der AMP-Architektur: Obwohl der Cortex-M7 durch komplexe Fließkommaberechnungen und langsame UART-Übertragungen massiv belastet ist, beeinflusst dies die harte Echtzeit des M4 überhaupt nicht. Die Kerne sind logisch und physikalisch entkoppelt. Das System hält seine Timing-Garantien ein.

Herzlichen Glückwunsch zum erfolgreichen Capstone-Projekt!